



Von der DevNet-Pipeline zum intelligenten Labor mit MCP

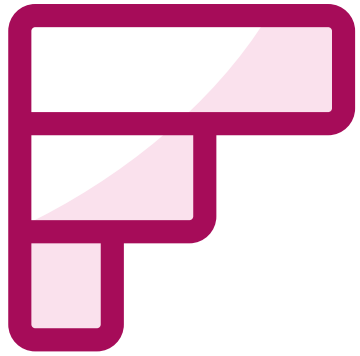
Akademietag 2026 in München



- 01 Rückblick - Partnermeeting in Thessaloniki (10 min)
- 02 Theorie und Praxisangebot: DevNet CI/CD – Das Konzept (15 min.)
- 03 Praxis: CML Lab mit KI automatisieren (45 min.)
- 04 Theorie und Praxisangebot: MCP - Ein Blick unter die Haube (35 min.)
- 05 Reflexion und Ausblick (15 min.)

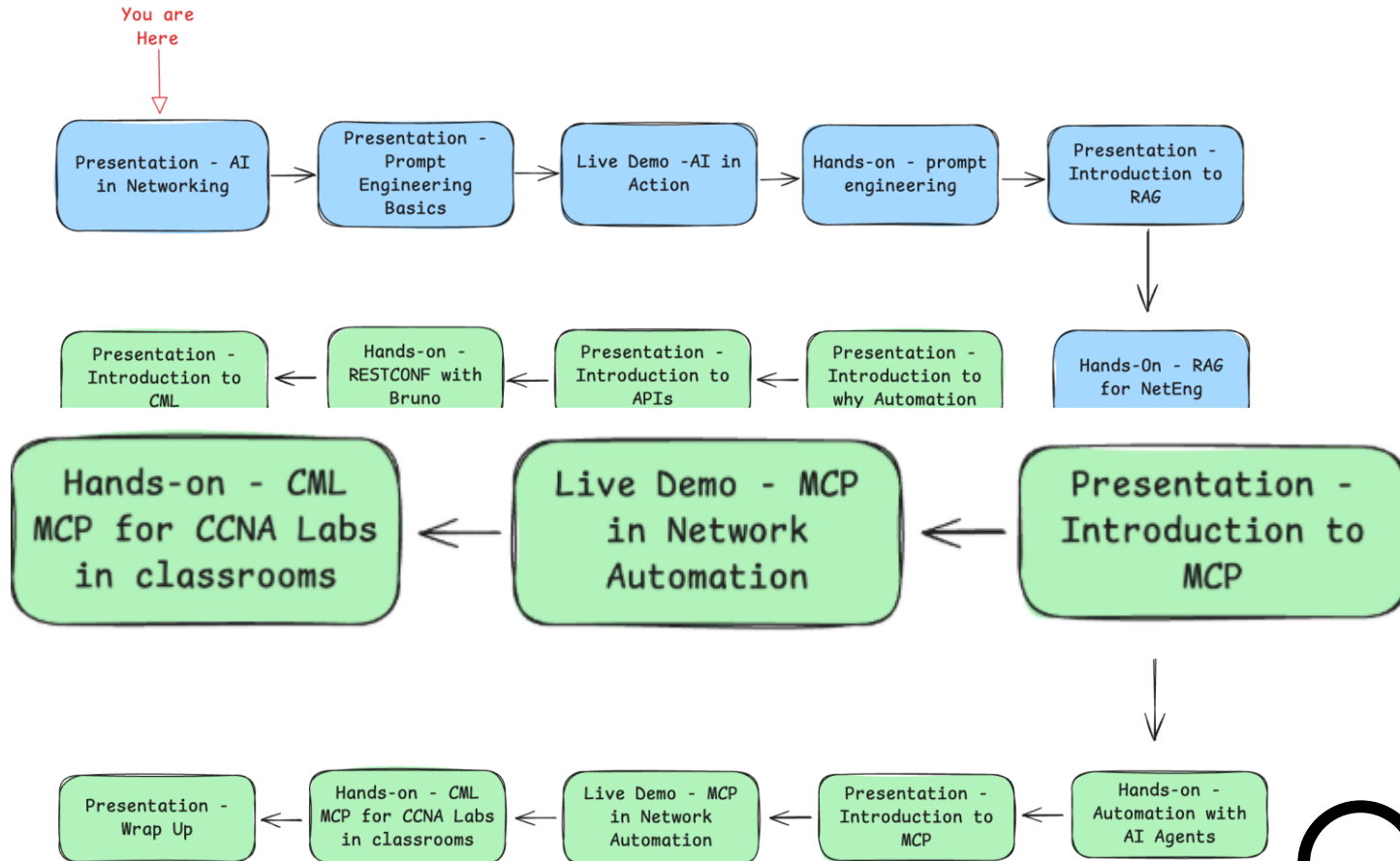


Welche Technologien verbinden Sie mit dem Thema Netzwerkautomatisierung?



**Mit welchen Werkzeugen der
Netzwerkvirtualisierung arbeiten Sie?**

Einfluss des Partner-Meetings 2025 in Thessaloniki Workshopteilnahme vom 9.-10.10.2025



Patrick Gargano, Quinn Snyder, Kareem Iskander,
Hank Preston, Alexander Stevenson

GitHub Repo

<https://github.com/CiscoLearning/CiscoU.-Workshop>





In Eigenregie

Claude Desktop (inkl. Account), Alternative: einen MCP-fähigen Host

- <https://claude.ai/download>

Bruno (Open-Source API Client), Alternative: Postman

- <https://www.usebruno.com/downloads>

Cisco Modeling Lab (CML) free 2.9, Alternative: GNS3 oder Eve NG

- <https://mkto.cisco.com/cml-free.html>

Python 3.11 oder höher, <https://www.python.org/downloads>

- Rest über requirements

Git, <https://git-scm.com/downloads>

CML-MCP-Server, <https://github.com/xorrkaz/cml-mcp.git>

Ollama <https://ollama.com/download> plus modell

- Alternative LM Studio, <https://lmstudio.ai/>

MCP Inspector, <https://modelcontextprotocol.io/docs/tools/inspector>

Wireshark, <https://www.wireshark.org/download.html>

Auf 12 ALP-Laptops vorinstalliert admin, 12345

Claude Desktop (bitte eigenen Account nutzen)

Bruno (Open-Source API Client)

Cisco Modeling Lab (CML) free 2.9 (admin, cml4free)

CML MCP-Server

Python 3.11

Git

Ollama mit qwen ...

MCP Inspector ...

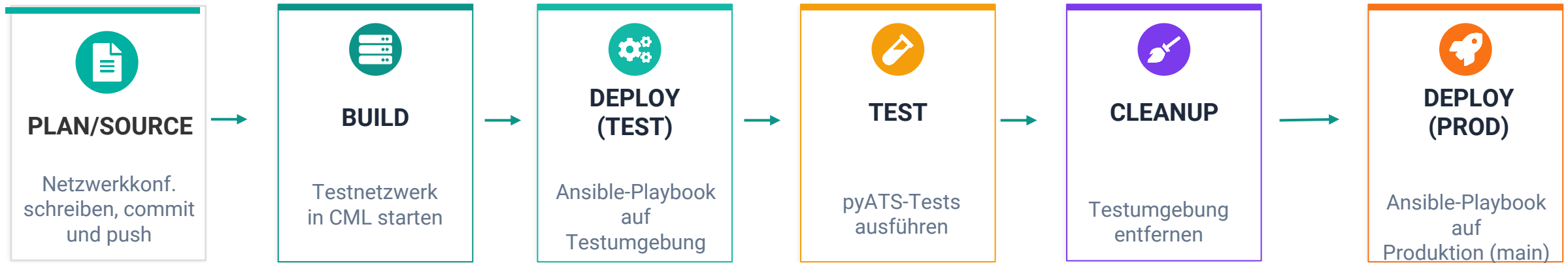
Wireshark



Praxisangebot: https://github.com/Lotter69/CI_CD_Kurs

Continuous Integration (CI)

Continuous Delivery (CD)



Branching-Strategie

test Branch: Commit → Build → Deploy:Test → Test → Cleanup (*CI: nur Testumgebung*)

main Branch: Merge → Deploy:Prod → Cleanup (*CD: Produktionsumgebung*)

Exemplarisches Vorgehen mit Ansible:

1. ACL-Konfig. kodifizieren (YAML)
2. Ansible-Playbook erstellen
3. Inventar (Test + Prod) anlegen
4. CML-Testskript (Python) entwickeln
5. pyATS-Test schreiben
6. CI/CD-Workflow definieren
7. Pipeline auslösen & validieren



Continuous Integration (CI)

 **Entwickler macht ACL-Änderung** → Commit & Push (Branch: test)

GitHub Actions – Automatische Test-Phase

- 1 **Start Test-Netzwerk in CML** – 2 virtuelle Router werden gestartet
- 2 **Deploy ACL-Konfiguration auf Test-Router** – Ansible wendet neue ACL an
- 3 **Automatische Tests (pyATS)** – Konnektivitäts- & Funktionsprüfung
- 4 **Cleanup** – Test-Netzwerk wird gelöscht

 **Tests erfolgreich!**

 **Pull Request: test → main** → Review & Merge 

Continuous Delivery (CD)

Automatisches Deployment nach Merge

- 1 **Ansible deployed ACL auf Produktiv-Router** – Konfiguration wird ausgerollt
- 2 **Verifizierung** – Deployment-Status wird geprüft

 **Produktiv-Umgebung ist aktualisiert!**

Praxisangebot: https://github.com/Lotter69/CI_CD_Kurs



Technologiestack (Beispiel)

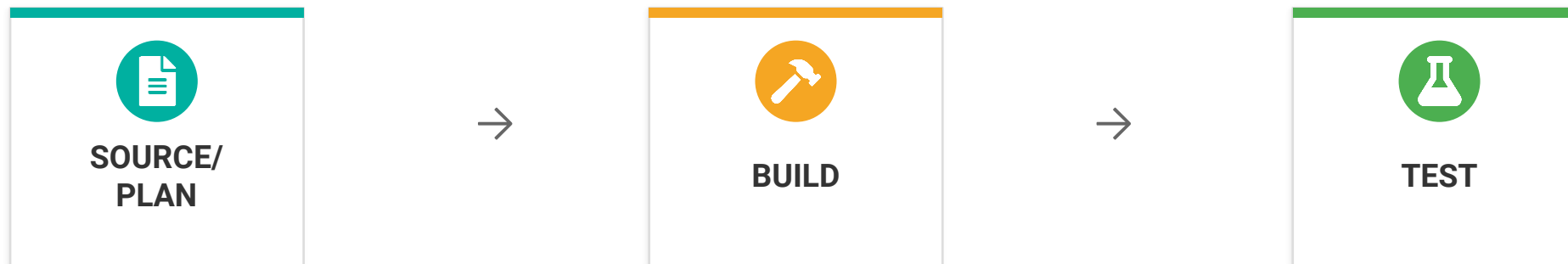
Versionskontrolle	GitHub • Git Branching (test / main)
CI/CD-Engine	GitHub Actions • CI/CD-Workflow • Runner
Testumgebung	Cisco CML • Python-Skript (CML-API)
Konfig.-Management	Ansible • Playbooks • YAML • Inventar
Validierung	Cisco pyATS • Testbed • Konnektivitätstests
Zielinfrastruktur	Cisco IOS Router (CSR1000v, IOL in CML)

Handlungskompetenzen (exempl.)

IaC-Grundlagen	Netzwerkconfiguration als Code erfassen (z. B. YAML)
Automatisierung	Playbooks schreiben
CI/CD-Pipelines	Mehrstufige Pipelines in CI/CD-Tool (z. B. github) definieren
Testautomatis.	pyATS-Testskripte erstellen, z. B. Konnektivität automatisiert validieren
Umgebungsmgmt.	CML-Testumgebungen per API programmatisch erstellen, nutzen und bereinigen
Git-Workflows	Branch-basierte Entwicklung, Merge Requests

Praxisangebot: https://github.com/Lotter69/CI_CD_Kurs

Was wäre wenn KI-Systeme in der Pipeline helfen könnte?



KI-Systeme könnten ...

SOURCE/PLAN

- Requirements lesen
(z.B. "CCNA Lab zum InterVLAN Routing")
- Topologie-Design vorschlagen

BUILD

- Labor-Topologie generieren
- Geräte-Konfigurationen erstellen

TEST

- Labor autom. in CML erstellen
- Tests generieren
- Ergebnisse bewerten

Das probieren Sie jetzt aus!



Schritt 1

Setup prüfen

10 Min

UV funktioniert?
Claude Desktop
konfiguriert?
MCP-Server läuft?



Schritt 2

Test-Labor erstellen

5 Min

Kleines Test-Labor zur
Verbindungsprüfung



Schritt 3

CCNA-Labor generieren

15 Min

CCNA-Blueprint hochladen
Prompt eingeben
Claude erstellt komplettes
Lab



Schritt 4

Labor erkunden

10 Min

In CML ansehen
Lab-Anweisungen lesen
Aufgaben durchgehen
Optional: Bewertung testen



Schritt 5

Reflexion

5 Min

Erkenntnisse?
Bezug zur DevNet-
Pipeline?

Auf den bereitgestellten Notebooks der ALP ist bereits alles installiert. (Infos im Ordner C:\akademietag_2026\mcp_cml)
- CML starten, Claude Desktop MCP konfig., Testlab erstellen, usw.

erforderliche Credentials:
CML: *admin, cml4free*
pyATS: *cisco, cisco, cisco*

Eigenregie:

Schritt für Schritt-Anleitung siehe github-Repo

https://github.com/Lotter69/CML_MCP

Praxis: CML Lab mit KI automatisieren (45 min.) Troubleshooting, häufige Stolpersteine



MCP-Server läuft nicht?

Claude Desktop komplett neu starten (Prozess im Task-Manager beenden und Neustart von Claude Desktop)

→ Settings → Developer → Server-Status prüfen

UV nicht gefunden?

→ Terminal neu starten

→ uv --version testen

CML-Verbindung fehlgeschlagen?

→ Credentials in config prüfen

→ CML-URL im Browser testen

Claude erstellt kein Labor?

→ Prompt neu formulieren

→ PDF wirklich angehängt?

→ Chat neu starten

Generell:

→ Handout Schritt-für-Schritt folgen

→ Fehlermeldungen komplett lesen

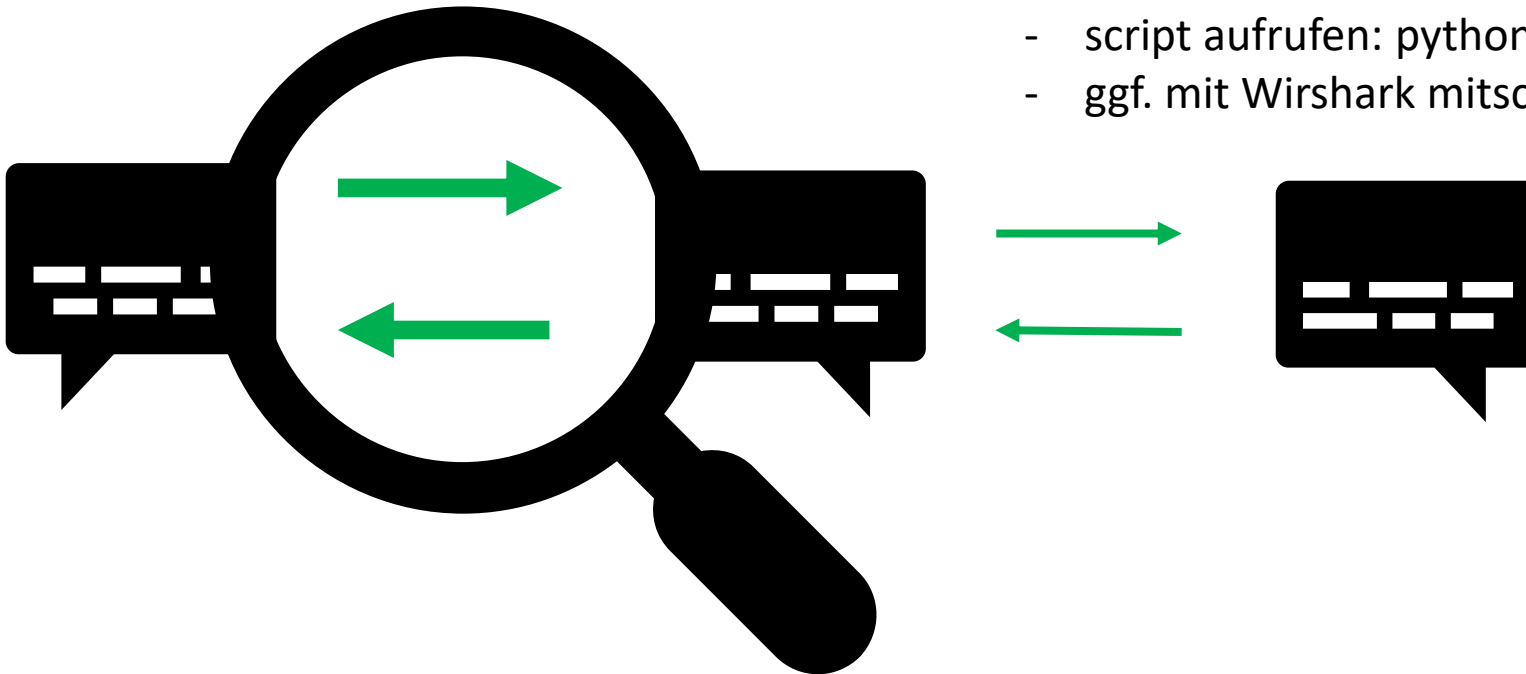
→ Michael oder Florian fragen!



Praxis-Angebot:

Auf den bereitgestellten Notebooks der ALP ist bereits alles installiert.
(Infos im Ordner C:\akademietag_2026\mcp_cml)

- venv starten: `.\venv\Scripts\Activate.ps1`,
- script aufrufen: `python einfacher_api_call_ollama.py`
- ggf. mit Wireshark mitschneiden



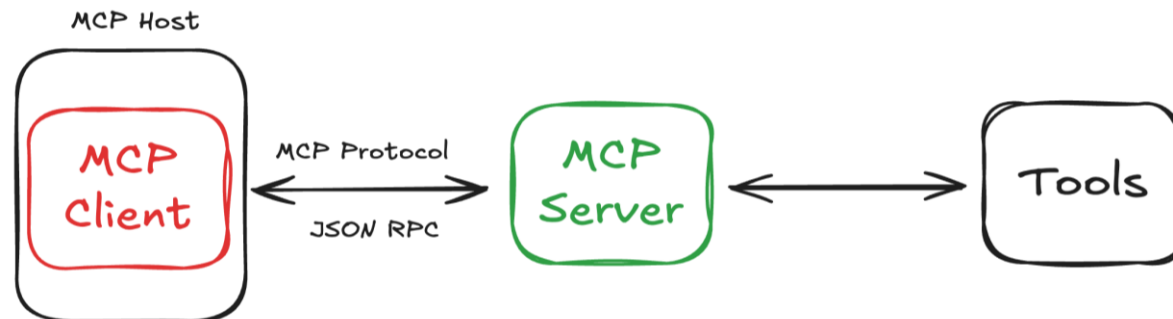
Schritt für Schritt:

Github Repo: https://github.com/Lotter69/mcp_basics



MCP Client

- Ist **im MCP-Hosts** (KI-App, IDE oder Agentenplattform) **enthalten**
- Kommuniziert mit **einem** oder **mehreren MCP-Servern**.
- Sendet Anfragen / empfängt Antworten
- Ein MCP-Host wird durch **Konfiguration (json-Datei)** MCP-Client-fähig



MCP Server

- Ist Brücke zwischen KI-Agenten und realen Diensten/Tools
- Stellt Funktionen (Tools, Ressourcen, Prompts) in JSON bereit.
- Führt Anfragen aus, die vom MCP-Client gestellt wurden
- Verbirgt Komplexität und bietet eine einheitliche MCP-Schnittstelle
- Wird vom Dienst-/Toolanbieter gewartet
 - es gibt auch [OpenSource-Community-Entwicklungen](https://www.anthropic.com/news/model-context-protocol)

<https://www.anthropic.com/news/model-context-protocol>



Verbindung	Protokoll (wie wird transportiert)?	Format (Wie ist der Transportinhalt aufgebaut: z. B. json, xml, json-rpc)	API (Welche Methoden/Endpunkte stehen zur Verfügung)
MCP-Client / MCP Server (lokal)	stdio	JSON-RPC 2.0	tool/call, tool/list, resources/read, resources/list, prompts/get, prompts/list
MCP-Client / MCP Server (remote)	HTTP/SSE	JSON-RPC 2.0	Siehe oben
MCP-Client / LLM (remote)	HTTPS	JSON (REST-API)	POST, GET, DELETE usw. und Ressource als URL
MCP-Client / LLM (lokal)	HTTP	JSON (REST-API)	POST, GET, DELETE usw. und Ressource als URL

<https://modelcontextprotocol.io/specification/2025-03-26/basic>

<https://modelcontextprotocol.io/specification/2025-03-26/basic/lifecycle>

<https://modelcontextprotocol.io/specification/2025-03-26/server>



MCP Server



Tool (Funktion)
hier hartcodiert

```
TOOL_DESCRIPTION = Tool(
```

name: Eindeutiger Identifier des Tools

- Wird vom Client verwendet um das Tool aufzurufen
- Sollte beschreibend sein

```
    name="patchkabel_bestand_abfragen",
```

```
    description="Fragt den aktuellen Lagerbestand von Patchkabeln ab. Zeigt Bestände von blauen und roten 1m Patchkabeln sowie den Gesamtbestand an.",
```

```
    inputSchema={
```

inputSchema: JSON Schema für Parameter

- Standard JSON Schema Format
- Definiert welche Parameter das Tool erwartet

description: Beschreibung für das LLM

- Erklärt dem LLM wann das Tool verwendet werden soll
- Sollte klar und detailliert sein
- LLM nutzt diese Info zur Entscheidung

```
        "type": "object",      # Hauptobjekt
        "properties": {       # Liste aller Eigenschaft.
```

```
            "farbe": {
                "type": "string",
                "description": "Die Farbe des Patchkabels (blau, rot, oder gesamt für alle)",
                "enum": ["blau", "rot", "gesamt"]
            }
        },
```

```
        "required": ["farbe"] # Pflichtangabe, hier die Farbe
```

```
    }
```



```
# Hardcodierter Lagerbestand
LAGER_BESTAND = {
    "blau_1m": 45,
    "rot_1m": 32
}
```



MCP Server

Tool (Funktion)
hier hartcodiert

```
def patchkabel_bestand_abfragen(farbe: str) -> str:
```

```
    """
```

```
    Tool-Funktion: Fragt Patchkabel-Bestand ab
```

```
    Args:
```

```
        farbe: Die Farbe des abzufragenden Patchkabels (blau/rot/gesamt)
```

```
    Returns:
```

```
        Formatierter String mit dem Bestandsinformationen
```

```
    """
```

```
    if farbe == "blau":
```

```
        bestand = LAGER_BESTAND["blau_1m"]
```

```
        return f"Bestand blaue 1m Patchkabel: {bestand} Stück"
```

```
    elif farbe == "rot":
```

```
        bestand = LAGER_BESTAND["rot_1m"]
```

```
        return f"Bestand rote 1m Patchkabel: {bestand} Stück"
```

```
    ....
```

Docstring: Beschreibt dem Entwickler was die Funktion macht.
Das LLM verwertet die Tool-Beschreibung.

Funktion/Programmlogik: Implementiert die eigentliche Funktionalität
hier hartcodiert
Alternativen: z. B. API-Call, SQL-Abfrage, ...



Benutzer

Startet Client
und gibt ein



**LLM
(API-Provider)**

“Denkt” & wählt Tool aus



Host / MCP Client

z.B. Claude Desktop



MCP Server

Stellt Tools bereit



Tool (Funktion)

Führt Aktion aus

INITIALISIERUNG

Startet Programm

`initialize()`

Server bereit ✓

`list_tools()`

`[patchkabel_bestand_abfragen]`

Was passiert:

- Client startet Server-Prozess (via stdio)
- Server initialisiert sich
- Client fragt verfügbare Tools ab
- Server sendet Tool-Liste zurück



Benutzer



LLM



Host / MCP Client



MCP Server



Tool (Funktion)

"Wie viele blaue Kabel haben wir?"

Was passiert:

- Benutzer stellt Frage
- Client empfängt Frage

BENUTZER-ANFRAGE

messages: "Wie viele blaue Kabel..." + tools: [patchkabel_bestand_abfragen]

AI-MODELL ENTSCHEIDUNG

LLM analysiert Anfrage

Entscheidet: Tool benötigt

Wählt Tool: patchkabel_bestand_abfragen

tool_use:

name="patchkabel_bestand_abfragen"
arguments={"farbe": "blau"}

Was passiert:

- Client sendet Anfrage und Tool-Liste an LLM
- LLM analysiert die Frage
- LLM entscheidet: "Ich brauche das Tool patchkabel_bestand_abfragen"
- LLM gibt Anweisung zurück: Tool ausführen mit Parameter "blau"



Benutzer



**LLM
(API-Provider)**



Host / MCP Client



MCP Server



Tool (Funktion)

1. BENUTZER → CLIENT

Input: "Wie viele blaue Kabel haben wir?"

2. CLIENT → LLM

```
Request: {  
  "model": "qwen2.5-coder:3b",  
  "messages": [{"role": "user", "content": "Wie viele blaue Kabel haben wir?"}],  
  "tools": [{  
    "name": "patchkabel_bestand_abfragen",  
    "description": "Fragt den aktuellen Lagerbestand...",  
    "input_schema": {...}  
  }]  
}
```

3. LLM → CLIENT

```
Response: {  
  "stop_reason": "tool_use",  
  "content": [{  
    "type": "tool_use",  
    "name": "patchkabel_bestand_abfragen",  
    "input": {"farbe": "blau"}  
  }]  
}
```



Benutzer



LLM
(API-Provider)



Host / MCP Client



MCP Server

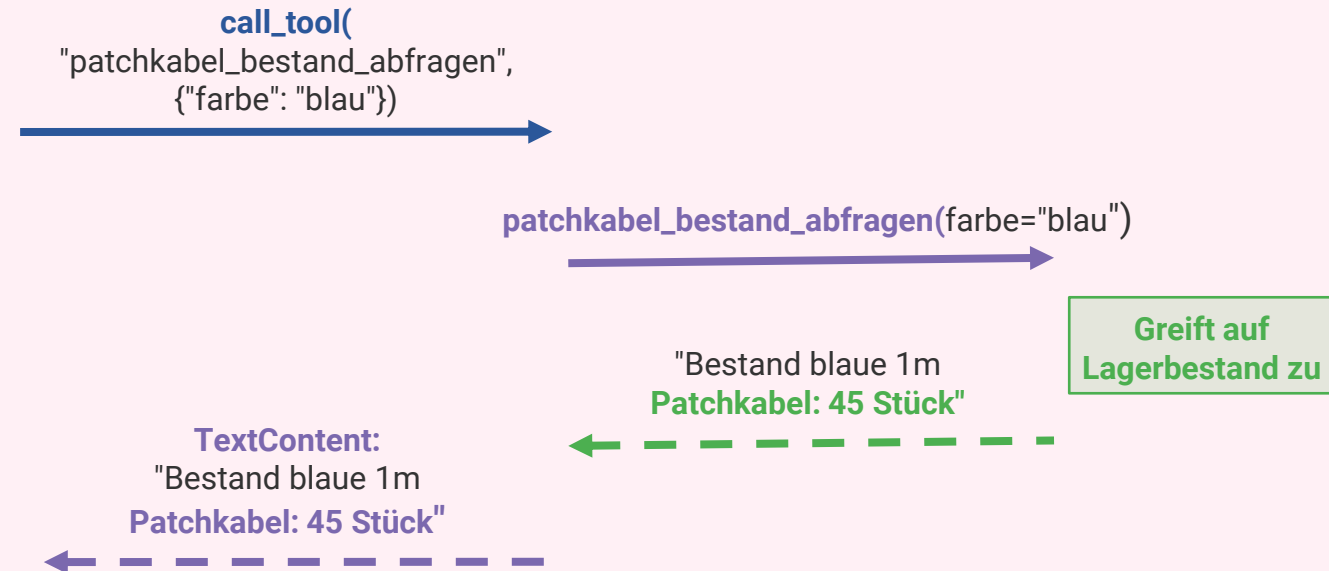


Tool (Funktion)

TOOL-AUSFÜHRUNG

Was passiert:

- Client ruft Tool auf Server auf
- Server ruft Python-Funktion auf
- Funktion greift auf Daten zu (LAGER_BESTAND)
- Funktion gibt Ergebnis zurück
- Server verpackt Ergebnis in TextContent
- Server sendet an Client





Request: Client -> Server (Schritt 4),

- ... kannst du bitte das Werkzeug mit dem Namen (name: xxx) und folgenden Einstellungen (arguments: xxx) nutzen



Host / MCP Client



MCP Server



Tool (Funktion)

Response: Server -> Client (Schritt 7)

- ... die Nutzung meines Werkzeugs hat zu folgendem Ergebnis geführt (content: xxx)

4. CLIENT → MCP SERVER

```
MCP Call: {  
  "method": "tools/call",  
  "params": {  
    "name": "patchkabel_bestand_abfragen",  
    "arguments": {"farbe": "blau"}  
  }  
}
```

5. MCP SERVER → TOOL

Python Call: patchkabel_bestand_abfragen(farbe="blau")

6. TOOL → MCP SERVER

Python Return: "Bestand blaue 1m Patchkabel: 45 Stück"

7. MCP SERVER → CLIENT

```
MCP Response: {  
  "result": {  
    "content": [{"type": "text", "text": "Bestand blaue 1m Patchkabel: 45 Stück"}]  
  }  
}
```

Funktionsaufruf z. B. hartcodiert, oder API Call oder SQL-Abfrage oder ...

- Schritt 5/Schritt 6



Benutzer



LLM
(API-Provider)



Host / MCP Client



MCP Server



Tool (Funktion)

ANTWORT-GENERIERUNG

tool_result:

"Bestand blaue 1m Patchkabel: 45 Stück"

**Formuliert
natürliche Antwort**

"Im Lager befinden sich
aktuell 45 Stück blaue
1m Patchkabel."

Was passiert:

- Client sendet Tool-Ergebnis zurück an LLM
- LLM liest das Ergebnis
- LLM formuliert natürliche Antwort basierend auf Daten
- LLM sendet finale Antwort an Client

AUSGABE

Antwort anzeigen:

"Im Lager befinden sich aktuell
45 Stück blaue 1m Patchkabel."

Was passiert:

- Client zeigt Antwort im Terminal an
- Benutzer sieht die Information



Nutzung der REST API von CML 90% (Automatisierung von CML-Umgebungen, keine Gerätekonfiguration)

- Labormanagement z. B.
 - Labs erstellen, starten, stoppen und löschen
- Node-Verwaltung z. B.
 - Virtuelle Geräte (Nodes) hinzufügen, konfigurieren und entfernen
 - Nodes starten, stoppen und neu starten
- Netzwerkkonfiguration z. B.
 - Verbindungen zwischen Nodes erstellen und löschen
 - Netzwerk-Interfaces konfigurieren
- Automatisierung z. B.
 - Tests und Szenarien automatisch ausführen
- Monitoring z. B.
 - Status von Nodes und Labs überwachen
- Benutzerverwaltung z. B.
 - Benutzer und Berechtigungen verwalten
- **Node-Konfiguration**
 - **nur über Startup Config, die beim Start des Nodes angewendet wird**

PyATS/Genie Integration (Erweiterung) 10%

- Eigenständiges Test-Framework
 - Erstellt PyATS Testbed aus CML Lab
 - **Verbindet sich direkt zu Geräten (SSH/Telnet)**
 - Führt CLI-Befehle aus
 - Parst Output mit Genie

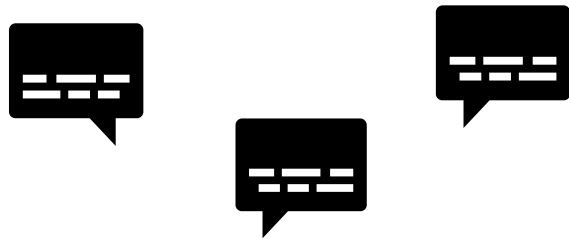
Praxis-Angebot:

Github Repo: https://github.com/Lotter69/CML_MCP/tree/main/CML-API-Collection

oder nutzen Sie den MCP Inspector (siehe lokalen Ordner „Akademietag 2026“)



Zitat: Fehler bei der Konfiguration und beim Änderungsmanagement verursachen mittlerweile fast 45 % aller netzwerkbezogenen Ausfälle – wodurch Konfigurationsabweichungen zu einer der größten Bedrohungen für die Verfügbarkeit und Ausfallsicherheit werden.



Welche Stärken sehen Sie in der MCP-Architektur?

Welche Schwachstellen könnte MCP eröffnen?

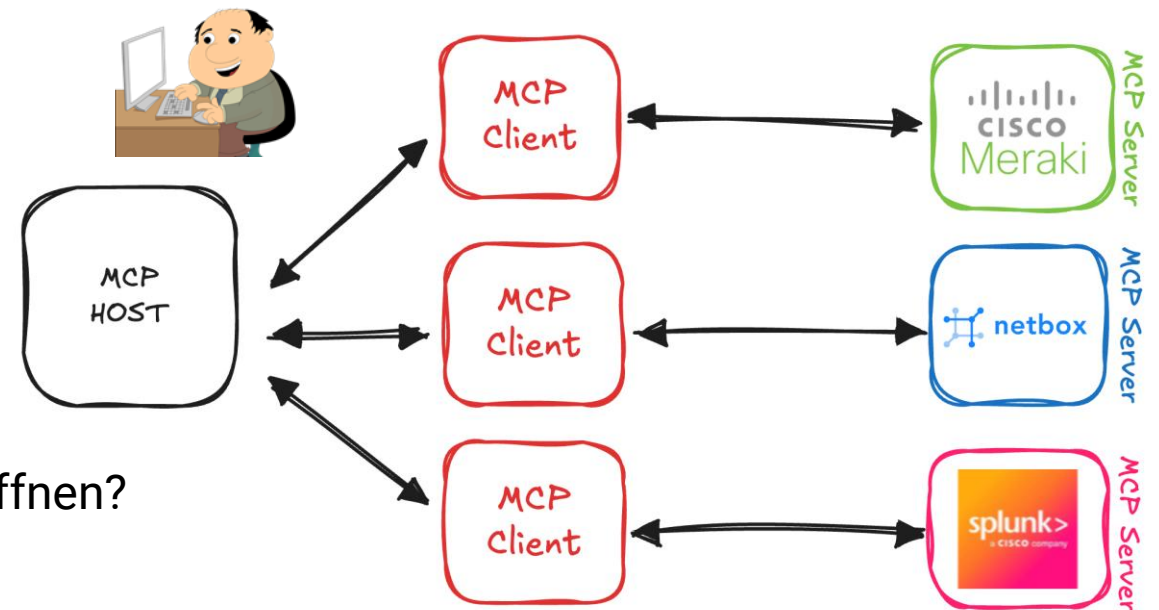
Ist MCP überhaupt notwendig?

Wie „scharf“ sind nun die Ergebnisse in der Konstellation LLM-MCP-CML?

Wo würden Sie MCP in Ihrem Unterricht einsetzen?

Wie bereiten Sie Fachinformatiker KI-DevOps vor?

Was können wir einem FISl aus dem Wissensgebiet NetDevOps zumuten bzw. was können wir befördern?





zu MCP

- Anthropic, <https://www.anthropic.com/news/model-context-protocol>
- LF Projects, LLC, <https://modelcontextprotocol.io/docs/getting-started/intro>
- Vorhandene MCP-Server, - <https://github.com/punkpeye/awesome-mcp-servers/>
- Baue deinen ersten MCP-Server – FastMCP – <https://gofastmcp.com/getting-started/welcome>

GitHub-Repos

- MCP Grundlagen erkunden - https://github.com/Lotter69/mcp_basics
- CI/CD-Kurs: https://github.com/Lotter69/CI_CD_Kurs
- Workshop_AI_Automation_2025 angepasst, https://github.com/Lotter69/Workshop_AI_Automation_2025
- CML API-Collection,
- Workshop Cisco U. original, <https://github.com/CiscoLearning/CiscoU.-Workshop>
- cml-mcp, <https://github.com/xorrkaz/cml-mcp>

Emerging Technologies Workshop

- Offene Gruppe, die sich max. 2x im Jahr für eine Woche trifft und erprobt, entwickelt, dokumentiert.
- Anspruch:
 - Ergebnisse münden in Unterricht und Lehrerfortbildung
- Themen:
 - Alles was „emerging“ für IT-Infrastruktur ist
- Nächster Workshop:
 - [15.06.-19.06.2026 an der ALP: 25-26.2_03314-1](#)
 - 14.12.-18.12.2026 an der ALP: 26-27.1_05627-1